DEPURACION DE MI BOMBA

- Para generar el ejecutable he usado:

gcc -Og bomba_jr.c -o bomba_jr -no-pie -fno-guess-branch-probability

- Para empezar con el análisis de la bomba con el fin de encontrar las contraseñas usamos:

gdb -tui bomba_jr

(gdb) layout asm //para ver el código en ensamblador

(gdb) layout regs //para ver las direcciones en las que guardan los contenidos de los registros

(gdb) br main

(gdb) run

- Primera parte, encontrar la contraseña:

Para empezar buscamos todas las llamadas a boom(), y seleccionamos aquellas donde se haga uso de <strncmp> ya que ahí se hará la comprobación de la contraseña, en mi caso:

(gdb) br *main+156

(gdb) continue

// insertamos cualquier palabra como contraseña (pj: hola)

(gdb) x/s $rdi

(gdb) x/s $rsi

Como podemos ver ninguno de los parámetros se parece a la contraseña que hemos intentado poner (rdi = jqnc...; rsi = ngocfgnupfykej). Podemos ver que si esta vez ejecutamos el programa e intentamos poner la contraseña como aparece en rsi, la bomba explota por lo que esta no es la contraseña.

Si miramos el código podemos ver como se ejecuta varias veces un bucle entre que el programa nos pregunta la contraseña y se hace la comprobación, por lo que podríamos pensar que tiene lugar una encriptación de la contraseña.

Podemos tomar dos rutas de acción:

1. Podemos ejecutarlo varias veces para ver si es posible discernir un patrón entre las distintas palabras. Incluso podemos introducir el contenido que ya vimos en rdi y en rsi para ver las diferencias entre distintas palabras.

Aunque esta opción no es muy recomendable dado que la encriptación podría ser muy complicada e imposible de resolver por fuerza bruta, no perdemos nada por intentarlo.

2. Podemos analizar el bucle que hemos visto antes para saber qué clase de operaciones se ejecutan sobre las palabras para encriptarlas.

Si este es el camino elegido el código a analizar es el siguiente:

main+135 cmp $0xf, %rax

main+139 jbe (...) <main+170>

main+170 addb $0x2, 0x30(%rsp, %rax, 1)

main+175 addb $0x2, 0x50(%rsp, %rax, 1)

main+180 add $0x1, %rax

main+184 jmp (...) <main+135>

El código tiene las siguientes características:

- Podemos ver que es un bucle que se ejecuta un total de 15 veces ya que empieza comparando 15 (0xf) con %rax y salta si es menor o igual a main+170.
- En main+170 y en main+175 se suma 2 el valor de dos posiciones con respecto al registro %rsp, las posiciones se calculan con respecto a rax por lo que seguramente sea un array de memoria.
- Por último, suma 1 a %rax y vuelve a comenzar el bucle con la primera comprobación.

Al tratarse de un bucle que modifica rsp tras pedirnos la contraseña y modificar dos arrays podemos entender que el encriptado se basa en sumar 2 a cada char de los dos arrays que resultan ser el array de la contraseña y el de la clave que nos pide el programa.

El resultado final de este análisis es que a cada char de la palabra se le suma dos posiciones en la tabla ascii. Con esto en mente ya podemos descifrar la clave que resulta ser "lemadelsandwich".

- Pasamos ahora a la segunda parte, el PIN:

Al igual que en la parte anterior buscamos las llamadas a boom(). Vemos que están en main+304 y en main+340, sin embargo en main+294 tenemos que se compara el contenido de 0xc(%rsp) con otro valor.

Usamos el siguiente código:

(gdb) p/d 0x1395 = (5013)

Sin embargo, vemos como anteriormente están las siguientes ordenes tras preguntarnos el pin:

main+298 add $0x5, %eax

Por tanto, el pin es igual a 5013-5 = 5008

Con esto ya habríamos terminado de desactivar la bomba.